

AI-Driven Scheme Matching Platform — Web App Requirements

SIH26092 | Ministry of Social Justice and Empowerment (MoSJE)

1. Functional Requirements (What the app must do)

Module A — Smart Scheme Recommender

- Multi-step intake form: project type, estimated cost, annual family income, education status, caste category proof status
- Rules engine that maps inputs → scheme (Micro Finance ≤ ₹1.4L / Term Loan ≤ ₹50L / Education Loan)
- Confidence/eligibility score display with plain-language explanation ("Why this scheme?")
- Multilingual support (minimum Hindi + English toggle)

Module B — Financial Calculator

- EMI calculator using standard formula: $EMI = P \times r \times (1+r)^n / ((1+r)^n - 1)$
- Scheme-specific interest rate lookup (6.5%–15%)
- Moratorium period handling (3–12 months, EMI start delay)
- Amortization schedule table + downloadable summary

Module C — Geo-Spatial Partner Locator

- Map view (Leaflet/Google Maps) showing nearby SCAs/Banks/NBFC-MFIs
- Distance-based sorting from user's location
- Filter by partner type and scheme category served
- Mock "eligibility score" per partner (clearly labeled as simulated for demo)

Module D — Auth & Profile (optional, for judging polish)

- Simple login/signup (or guest mode to skip friction in demo)

- Saved application history / scheme comparison
-

2. Non-Functional Requirements

Requirement	Target
Responsiveness	Mobile-first, works on low-end Android browsers
Load time	< 3s on 4G
Offline resilience	Basic caching (PWA) for form/calculator, since target users may have poor connectivity
Accessibility	Large fonts, icon-based navigation for low-literacy users
Language support	At least 2 Indian languages
Data privacy	No storage of sensitive income/ID data beyond session unless explicit consent

3. Tech Stack Recommendation

Frontend

- **Framework:** React.js (or Next.js if you want SSR + easy deployment)
- **Styling:** Tailwind CSS (fast to build clean UI under time pressure)
- **Maps:** Leaflet.js + OpenStreetMap (free, no API key hassles) — or Google Maps JS API if you have a key
- **i18n:** `react-i18next` for multilingual toggle
- **Charts:** Chart.js or Recharts for amortization schedule visualization

Backend

- **Runtime:** Node.js + Express, or Python + FastAPI (FastAPI is great if you want to bolt on ML later)
- **Rules engine:** Plain conditional logic or a lightweight decision-table library — no ML model required for MVP
- **API design:** REST endpoints — `/recommend`, `/calculate-emi`, `/nearby-partners`

Database

- **PostgreSQL or MongoDB** for:
 - Scheme master data (rules, interest rates, limits)
 - Channel partner directory (name, location, type, category served, mock eligibility score)
 - User applications (if you add save/history)
- For hackathon speed: **SQLite** or a **simple JSON/CSV file** can substitute if time is short

Hosting/Deployment

- **Frontend:** Vercel or Netlify (free, instant deploy from GitHub)

- **Backend:** Render, Railway, or Fly.io (free tier sufficient for demo)
 - **Database:** Supabase (Postgres) or MongoDB Atlas (free tier)
-

4. Data You Need to Prepare Before/During the Hackathon

1. **Scheme rules table** — you'll need to research and hardcode actual scheme parameters (income caps, loan limits, interest rate bands, moratorium periods) from MoSJE/NBCFDC public documentation.
 2. **Channel partner directory** — curate a sample CSV of ~20–50 real or representative SCA/bank branches (name, lat/long, type, scheme category) for your demo region — full national coverage isn't needed for a hackathon.
 3. **Translation strings** — prepare a JSON dictionary of UI text in Hindi (and optionally a regional language relevant to your team/judges).
-

5. APIs / External Services Needed

Service	Purpose	Cost
OpenStreetMap / Leaflet	Map rendering	Free
Google Maps Geocoding API (optional)	Convert user address → lat/long	Free tier available
Google Translate API (optional)	Auto-translate if you don't hardcode strings	Free tier limited

6. Suggested Team Role Split (for a typical 4–6 person SIH team)

- **1–2 Frontend devs:** React UI, forms, calculator, map integration
 - **1 Backend dev:** rules engine, EMI calculation logic, API endpoints
 - **1 Data/Research person:** scheme rules research, partner directory curation, content accuracy
 - **1 Design/PPT person:** UI polish, pitch deck, demo script
 - **1 (optional) ML/NLP person:** if you want to add a chatbot layer or smarter recommendation scoring for differentiation
-

7. Suggested Build Timeline (36–48 hr hackathon)

1. **Hours 0–4:** Finalize scheme rules data, wireframe UI, set up repo/boilerplate

2. **Hours 4–14:** Build Scheme Recommender + Financial Calculator (core logic first, since these are your safest, fully-functional wins)
 3. **Hours 14–24:** Build Partner Locator with map integration + sample data
 4. **Hours 24–32:** Multilingual pass, styling polish, mobile responsiveness
 5. **Hours 32–40:** Testing, bug fixes, deploy to Vercel/Render
 6. **Hours 40–48:** Pitch deck, demo rehearsal, prepare answers for the "how would this scale / integrate with real NPA data" question
-

8. Anticipated Judge Questions (prepare answers)

- *"How do you get real-time partner NPA/eligibility data?"* → Be upfront: simulated for demo, production version would integrate with NABARD/SCA MIS or bank APIs.
- *"Is this actually AI, or just a rules engine?"* → Frame honestly: rules engine ensures reliability/explainability for a financial-eligibility use case (important for trust); optionally mention a roadmap to ML-based scoring using historical disbursement data for continuous improvement.
- *"How do you handle users with no smartphone/internet?"* → Mention PWA offline mode + potential SMS/IVR channel as a future extension.

